

Lab Procedure

Color Sensor

Introduction

Ensure the following:

1. You have reviewed the [Application Guide – Color Sensor](#)
2. Make sure the **Mechatronic Sensors Trainer** is out of its case; it is powered using the provided USB cable connected to the PC.
 - a. The LED next to the USB C port will stay white after the touch screen starts loading.
 - b. The load screen with the Quanser Logo should disappear after a few seconds and you should see some evenly spaced crosses on a black background on the touch screen.

Explore Raw Sensor Behavior

1. Launch Visual Studio Code or your preferred Python IDE. If using Visual Studio Code, click on **File > Open Folder...**, and browse to the directory containing the python files and lab procedure (`../sensors_trainer/1_fundamentals/color_sensor/hardware/python`). Open this directory.
2. Open `read_sensor.py`. Ensure that the `totalTime` variable is set to 50 seconds. Run this script using the  button or the terminal. This will open one plot **Color Sensor** and one window **LED Control**. The sensor signals are also printed in the terminal.
3. Move the sliders in the **LED Control** window and verify that LEDs in the Color Ambient Light Sensor cutout changes, and that the color sensor responds to the changes accordingly. When the script terminates after specified `totalTime`, you can drag the plot in **Color Sensor** to review previous data.
4. Take a folded white paper and firmly cover the color sensor cut out and run `read_sensor.py` again. With the RGB values set to 0, ensure no external room light leaks into the sensor and verify that all sensor readings drop near zero.
5. Set the Blue and Green sliders to 0 and slowly increase the **Red** LED slider from 0 to 255 and observe the live plot of sensor signals. Which sensor increases the most? How much does the **Clear** channel increase relative to others (RGB-to-clear ratio)? Do you observe any saturation in the signals? Record your observations.
6. Repeat step 4 and 5 for **Green** and **Blue** LED and record your observations.
7. Experiment with combination of **Red** and **Green** LEDs. Uncomment line #44, setting `gValue = rValue`, such that you can use the red slider to command equal R and G LED intensity. Record your observations on the RGB to Clear ratios, what do you notice?

Commented [LG1]: Do we wanna change the way we plot things so the red- green - blue line match rgb and then the grey line is clear? It was very confusing to me to understand what was going on until I ran the file like 3 times

Commented [LG2]: Is this the set tracker or the get tracker line?

Commented [LG3R2]: I added a line so that the observer can run from any folder. I'd put after which line you want this since I am not sure which does it and maybe in the code already add a `#gValue = rValue` and tell them to go uncomment that line

- Repeat step 7 for **Cyan (G and B)**, **Magenta (R and B)**, modify line #44 accordingly and record your observations.
- Command the LED to produce a white light. You can set **gValue = bValue = rValue** to control all 3 LEDs with one slider. Slowly increase the LED intensity. When the clear channel reaches ~1.0 (just below saturation), record the **LED intensity** (values of the sliders). This is your white reference intensity.
- Navigate to "Estimate Color" section near the end of the main loop and uncomment the lines beginning with **# rawRGBEstimate = np.zeros((200,200,3))**. This section will generate two color estimates, one based on the raw RGB data and the other based on the RGB-to-Clear ratios.
- Run the script again after uncommenting and a new window named **Color Estimate** will open, showing raw RGB-based estimate on the left and color ratio-based estimate on the right. While covering the color sensor with paper, adjust the LED sliders, and observe how each estimation method behaves. What are the main differences between each estimation methods?

Commented [LG4]: Is this LED intensity in 0-255 values? Or the one that gets printed in a 0-1 value?

Calibration

In this section, you will use the LEDs as approximate monochromatic light sources to calibrate the color sensor. By using lights with known wavelengths, we can determine how the **RGB** readings from color sensor values relate to the **XYZ** tristimulus values, which is a physically meaningful, device-independent definition of color. The goal of this section is to compute a linear mapping matrix **M** that converts the sensor's raw readings into XYZ values.

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} M_1 & M_2 & M_3 \\ M_4 & M_5 & M_6 \\ M_7 & M_8 & M_9 \end{bmatrix} \times \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

Refer to [Application Design Guide for VEML3328 \(designingveml3328.pdf\)](#) for computation of XYZ values (page 6), the formulation of M (page 8), and the calibration process (page 9). Because we do not have a lux meter, we will approximate true XYZ values using LED peak wavelengths.

- Using **read_sensor.py**, set the LED to emit only red light at the white reference intensity recorded earlier. Record several sensor readings, compute the average, and record it.
- Repeat the last step for green and blue LED as well. You now have a 3x3 matrix of sensor responses.

$$S = \begin{bmatrix} R_{LED-R} & R_{LED-G} & R_{LED-B} \\ G_{LED-R} & G_{LED-G} & G_{LED-B} \\ B_{LED-R} & B_{LED-G} & B_{LED-B} \end{bmatrix}$$

For example, R_{LED-B} represents the sensor signal in the red channel for blue LED light.

- In the same directory, open **calibrate.py**. In the main calibration function, fill in **sensorReadings** with values you measured. Then fill in **rgbWavelengths** using the

Commented [LG5]: I'd add something to include what I sent on slack to get color ratio with dividing by clear as an initial test, I think that one is a bit better if the object is not super close to the color sensor, but I'll leave it up to you.

Commented [LG6]: I know the concept review kinda talks about this but maybe tell them again, since I don't know what the monochromatic sources are or tristimulus values. Like a quick 1 or 2 sentences, and tell them to refer to the concept review for more info.

Commented [LG7R6]: As in, why are we doing this instead of using the data directly?

Commented [LG8]: I'd add RLED is the rgb values when you turned on the red LED or something so I know where to put which if that makes sense, the python file does show this which is good

information of peak wavelength emitted by each of the 3 colors found in the datasheet of the LED (**AAAF5050-MC-K12**).

15. Navigate to the helper function **wavelength_to_xyz()**. Complete this function using standard CIE 1931 color matching function provided in the design guide. Using this function, compute the XYZ values for the 3 LED wavelength. This gives you an estimated XYZ matrix.

$$T = \begin{bmatrix} X_{LED-R} & X_{LED-G} & X_{LED-B} \\ Y_{LED-R} & Y_{LED-G} & Y_{LED-B} \\ Z_{LED-R} & Z_{LED-G} & Z_{LED-B} \end{bmatrix}$$

16. Similar to the calibration process in the Design Guide, use the following equation to compute the mapping matrix, $M = T \cdot S^{-1}$. Make sure to construct **S** and **T** the same way as described in step 11, where Each LED's data forms a *column* of the matrix (not a row). This is different from **sensorReadings** and **xyzValues**, so double-check that each column corresponds correctly to Red, Green, and Blue LED measurements. Complete the **main** function and run **calibrate.py**, the mapping matrix will be printed in the terminal.
17. In the same directory, open **read_color.py**, and in the experiment parameters setup region, paste you mapping matrix **M**. Run the script and a window named **solid color** will open. Use the 3 sliders to control the "source color" on the left half. On the next step you will read this color and the color detected by the color sensor will be displayed on the right half.
18. Pick up the Sensor Trainer and press the color sensor against your computer monitor over the "source color" on the left. Move the sliders around and observe how the detected color respond to the source color. Does the detected color match the source color? How well does the detected color respond to the changes in the source color? Are there any biases in the colors? Record your observations.
19. Slightly move the sensor away from the screen. Does the mapping still behave consistently? Does distance affect the detected color? Record your observations and comment on whether XYZ-calibration approach seems robust for off-screen color measurement.

Commented [LG9]: What does this mean? To choose a different color or to try to match what is detected?

Color Matching

20. An alternative approach to convert sensor signals to sRGB values is to derive a direct mapping between the sensor signals and RGB values, bypassing XYZ entirely.

$$\begin{bmatrix} R_{screen} \\ G_{screen} \\ B_{screen} \end{bmatrix} = M \begin{bmatrix} R_{sensor} \\ G_{sensor} \\ B_{sensor} \end{bmatrix}$$

In the same directory, open **color_match.py**. In the experiment parameters region, fill in **screenColors** variables with at least 3 RGB values (integer between 0-255). More colors and more diversity lead to more robust linear fitting.

Commented [LG10]: Do we want them to be very different? Higher numbers for more bright colors? Also tell them if 0-1 or 0-255

21. Run this script and a window named **solid color** will open. Before continuing, resize or move your Visual Studio Code window so that the solid color window remains visible when you click back into the terminal. Press the color sensor against the window and press "Enter" in the terminal to continue the script. Specified color will be displayed in sequence, and sensor signals will be printed in the terminal. Verify that the sensor signals respond to different color on the monitor.
22. Navigate to **linear_fit** function, it will be used to compute the color matching matrix using collected sensor data and its corresponding screen color RGB values, stored in the **results** variable. Use off-the-shelf modules such as **np.linalg.lstsq** and **scipy.linalg.lstsq** and complete this function.
23. Run the script again. The mapping matrix **M** will be printed in the terminal.
24. Open **read_color.py** again and replace the existing **M** in the experiment setup region with the direct mapping matrix. Modify the main sensing loop region such that **sRGBDetected** is the outcome of $M \times RGB$.
25. Run the script and press the color sensor against the "source color". Move the sliders and compare the detected color with the source color. if the mapping is poor, return to the start of this section and redo the experiment including more colors.
26. Does the detected color match the source color? How well does the detected color respond to the changes in the source color? How is the performance of the direct mapping matrix comparing to XYZ-based method? Record your observations.
27. Slightly move the sensor away from the screen. Does the mapping still behave consistently? Does distance affect the detected color? Record your observations and comment on whether this direct-mapping approach seems robust for off-screen color measurement.
28. Stop, save and close your program.
29. Power OFF the Mechatronic Sensors Trainer by disconnecting its USB C cable, disconnect any external sensors and pack them along the base and the USB cables in the provided case.

Commented [LG11]: Tell them to make their vs code smaller so that the window does not disappear when they click back in the terminal lol

Commented [LG12]: The code in option 2 says and bias b to get values, what is that bias, I don't see it used?

Commented [LG13]: I'd add a question of what if they are a little more away from the color or if its not a screen but a colored object they have, to turn on the lights and read if its something worth exploring, I am not sure if it would work